A
Project Report
On

"CHARUSAT Online Course Assistant: A Retrieval-Augmented Generation Chatbot for Online Degree Programmes"

Prepared by

Stud Name (ID) <All>

Under the guidance of

Prof./Dr. Name of Internal Guide

A Report Submitted to
Charotar University of Science and Technology

for Partial Fulfillment of the Curriculum Requirements of

Bachelor of Technology

in Artificial Intelligence and Machine Learning

(5th Semester Project-1–AIUP301)

Submitted at



Department of Artificial Intelligence and Machine Learning

Chandubhai S. Patel Institute of Technology

At: Changa, Dist.: Anand – 388421

November 2026

CANDIDATE'S DECLARATION

I/We hereby declare that the project entitled "**CHARUSAT Online Course Assistant: A Retrieval-Augmented Generation Chatbot for Online Degree Programmes**" is my/our own work conducted under the guidance of **Prof. / Dr. Name of Internal Guide** and **Mr./Ms. Name of External Guide (if applicable)**.

I/We further declare that to the best of my/our knowledge, the project for B. Tech does not contain any part of the work, which has been submitted for the award of any degree either in this University or in other University without proper citation.

Student Name
(Student ID)

CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

CERTIFICATE

This is to certify that the report entitled "CHARUSAT Online Course Assistant: A Retrieval-Augmented Generation Chatbot for Online Degree Programmes" is a bonafied work carried out by Name of Student (Student ID) under the guidance and supervision of Prof./Dr. Name of Internal Guide for the subject Project -1 (AIUP301) of 5th Semester of Bachelor of Technology in Artificial Intelligence and Machine Learning at Faculty of Technology & Engineering – CHARUSAT, Gujarat.

To the best of my knowledge and belief, this work embodies the work of candidate himself/herself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred to the examiner.

Dr. Nirav Bhatt
Head of Department
Department of Artificial Intelligence and
Machine Learning
CSPIT, Changa, Gujarat.

Chandubhai S Patel Institute of Technology

At: Changa, Ta. Petlad, Dist. Anand, PIN: 388 421. Gujarat

ABSTRACT

Educational institutions receive a large number of repetitive questions about admissions, eligibility, fees, examinations and student support. The Center for Online Education (CCOE) at Charotar University of Science and Technology (CHARUSAT) offers online BBA, BCA, MBA and MCA programmes, and prospective learners need immediate, trustworthy answers drawn from official documents. This project designs and implements the CHARUSAT Online Course Assistant, a full-stack retrieval-augmented generation (RAG) chatbot that answers questions only from a curated knowledge base of official web pages, programme project reports and academic policies.

The system combines FastEmbed semantic embeddings with BM25 keyword search and reciprocal rank fusion, applies a relevance threshold, and sends the best six chunks to a grounded answer pipeline. Google Gemini is the primary language model, Groq provides high-speed failover, and a deterministic local extractor acts as the final fallback. Conversation context, programme clarification carry-forward, official contact routing, citation checks, numeric validation and honest refusal behaviour protect against unsupported answers. The React frontend provides streaming responses, authentication, guest mode, conversation history and an administration console, while FastAPI and MongoDB provide the API and persistence layers. On a 77-question benchmark, accurate answers increased from 64.9 percent to 75.3 percent and the 90th percentile latency fell from 24.6 seconds to 0.836 seconds. The final offline suite passes 56 backend tests, 12 frontend unit tests and 11 browser tests. The report documents the architecture, implementation, measured results, limitations and future extensions of the system.

ACKNOWLEDGEMENTs

I/We have taken efforts in this project. However, it would not have been possible without the kind support and help of many hands. I/We would like to extend my sincere thanks to all of them.

I/We am highly indebted to Internal Guide name for his/her guidance and constant supervision as well as for providing necessary information regarding the project & also for his/her support in completing the project.

I/We would like to express my gratitude to H.O.D. Dr. Nirav Bhatt and I am also grateful to all my faculty members of Chandubhai S. Patel Institute of Technology for their kind cooperation and encouragement which help me in completion of this project and preparing the report.

Last but not the least, I would also like to thank my colleagues, who have co-operated during the preparation of my report and without them this project has not been possible. Their ideas helped me a lot to improve my project report.

“We may not achieve everything we dream, but we cannot achieve anything unless we dream.” –

Name (ID)

List of Figures

Figure 1:	Layered Architecture of Computing Paradigm.	8
Figure 2:	Process flow of the LP-DRAM architecture for hypothesis1	18
Figure 3:	(a) Fog services count for three categories of processing devices with fixed nodes type for each fog service execution (Existing work by Xu et al., 2018) (b) . Fog services count for three categories of processing devices without fixed nodes (Hy-1)	23
Figure 4:	(a) Analysis of average consumption of resource by existing approach vs Proposed technique along with diverse statistics and without fixed nodes	23
Figure 5:	Resource Utilizations for (500,1000,1500,2000) services dataset	24
Figure 6:	Comparison of resource use using various datasets using the FF, BF, FFD, BFD, DRAM, and TAT methods	27

List of Tables

Table 1:	Comparison of current fog computing surveys	12
Table 2:	Simulation parameters	40
Table 3:	On-Demand Cost	41
Table 4:	Datasets used for experiments.	47
Table 5:	Reservation Cost (All upfront & 1 Year)	49

TABLE OF CONTENTS

LIST OF FIGURES 1
LIST OF TABLES 2
CHAPTER 1: INTRODUCTION 3
1.1 Overview 3
1.2 Problem Definition 4
1.3 Motivation 4
1.4 Research Objectives and Scope 5
CHAPTER 2: LITERATURE REVIEW 7
2.1 Background Study / Related Work 7
2.2 Summary of Existing Techniques 8
2.3 Research Gap Identification 9
2.4 Problem Formulation 10
CHAPTER 3: METHODOLOGY 11
3.1 Overview of the Proposed Work 11
3.2 System / Framework Design 11
3.3 Methodology / Workflow 13
3.4 Tools, Technologies, or Environment Used 17
CHAPTER 4: IMPLEMENTATION AND RESULTS 18
4.1 Implementation Details 18
4.2 Performance Evaluation Metrics 21
4.3 Result Analysis and Discussion 23
4.4 Comparison with Existing Methods 25
CHAPTER 5: CONCLUSION AND FUTURE EXTENSIONS 26
5.1 Summary of Contributions 26
5.2 Limitations 26
5.3 Future Extensions 27
5.4 Conclusion 28
BIBLIOGRAPHY 29

LIST OF FIGURES

Figure 3.1: End-to-end system architecture of the CHARUSAT Online Course Assistant 12

Figure 3.2: End-to-end query workflow with clarification, retrieval and refusal branches 14

Figure 4.1: Answer quality and latency before and after the final optimisation round 23

LIST OF TABLES

Table 1.1: Online degree programmes covered by the assistant 6
Table 2.1: Comparison of chatbot design approaches 8
Table 3.1: Technology stack 13
Table 3.2: Chunking configuration 15
Table 3.3: Development, testing and deployment environment 17
Table 4.1: Knowledge base composition and indexed chunks 18
Table 4.2: Principal backend modules 18
Table 4.3: Principal frontend modules 19
Table 4.4: Public and protected API endpoints 20
Table 4.5: Benchmark answer quality on 77 questions 21
Table 4.6: End-to-end latency 22
Table 4.7: Component and stress-test results 22
Table 4.8: Automated test and build results 22
Table 4.9: Before and after improvements 23
Table 4.10: Question bank coverage 24
Table 4.11: Comparison with existing approaches 25

CHAPTER 1: INTRODUCTION

1.1 Overview

Charotar University of Science and Technology (CHARUSAT) is a private university in Changa, Gujarat, and its Center for Online Education (CCOE) offers four online degree programmes: Bachelor of Business Administration (BBA), Bachelor of Computer Applications (BCA), Master of Business Administration (MBA) and Master of Computer Applications (MCA). The programmes are designed for learners who cannot attend a regular campus and who need flexible, technology-enabled education. Information about these programmes is published across several sources, including the official CCOE website, programme project reports submitted to the University Grants Commission, academic regulations, fee refund policies, examination rules and contact directories.

Prospective students, current learners and parents frequently ask the same set of questions: What is the fee structure? Am I eligible with my degree? How many semesters does the programme have? How do I appear for examinations? Who should I contact for LMS support? The answers exist, but they are distributed across long documents and pages. A general-purpose chatbot can produce fluent text, yet it may also invent fees, eligibility rules or deadlines. For an academic institution, an incorrect answer is worse than no answer because it can mislead a student about money, admission or examination eligibility.

This project addresses that problem with a retrieval-augmented generation (RAG) assistant. RAG is a design pattern in which the system first searches a trusted document collection and then asks a language model to answer using only the retrieved evidence [1]. The CHARUSAT Online Course Assistant applies this pattern to the CCOE knowledge base. It retrieves relevant passages from official documents, generates a grounded answer, attaches the source information for internal audit, and refuses to answer when the required fact is absent. The system is delivered as a web application with a React frontend, a FastAPI backend, a ChromaDB vector store and a MongoDB database for users, conversations and feedback.

The assistant supports all four programmes, English and Hinglish queries with common spelling errors, multi-turn follow-ups, programme clarification, authenticated conversation history, guest access with a message limit, as-you-type question suggestions and an administration console for monitoring knowledge gaps. Its behaviour is deliberately conservative: when documents conflict, the assistant reports the conflict; when a number is

not in the retrieved context, the assistant does not fabricate it; and when a question is outside the university domain, the assistant gives a short scope refusal and points the user to official contacts.

1.2 Problem Definition

The core problem is to provide fast, accurate and verifiable answers about the four CCOE online programmes without allowing the language model to rely on its own parametric memory. The project must satisfy several practical constraints at the same time.

- The answer must be grounded in official documents, not in the model's general knowledge.
- The system must handle exact course codes, programme names, fee figures and policy terms, which pure vector search often retrieves poorly.
- The system must understand follow-up questions and preserve the active programme and topic.
- The system must ask for a programme when a question such as 'What is the fee structure?' cannot be answered without one, and must carry the original question forward after the reply.
- The system must never expose student records, internal file paths or personally identifying information that may appear in admission rosters.
- The system must remain usable on a free cloud tier with limited memory and frequent cold starts.
- The system must be maintainable by a small student team, with clear modules, offline tests and a reproducible indexing pipeline.

A conventional FAQ page fails the first requirement because it cannot answer paraphrases. A fine-tuned model fails the second and third because factual knowledge is frozen in its weights. A naive RAG implementation improves grounding but still struggles with exact keywords and numeric facts. The project therefore formulates the problem as a hybrid retrieval and grounded generation task with explicit safety checks.

1.3 Motivation

The motivation for the project comes from both student experience and institutional workload. Admissions and student-services staff answer repetitive questions throughout the year, while prospective students expect immediate responses at any hour. An assistant that is

available 24 hours a day can reduce this load and provide consistent answers. However, the assistant must be trustworthy. The project is motivated by the following practical goals.

- Availability: give prospective and current students instant answers without waiting for office hours.
- Consistency: use one curated source of truth so that different users receive the same policy answer.
- Transparency: keep the retrieved evidence and source metadata so that every answer can be audited.
- Inclusiveness: tolerate typos, short forms and Hinglish phrasing commonly used by students.
- Safety: refuse unsupported questions and route users to the correct official department.
- Learning value: demonstrate a complete applied AI system that combines retrieval, generation, web engineering, databases, security and evaluation in one project.

The project also serves as a realistic study of the engineering trade-offs behind RAG. During development, several assumptions had to be measured rather than guessed: a cross-encoder reranker reduced accuracy, PyTorch embeddings exceeded the memory limit of the free hosting tier, and reciprocal rank fusion compressed relevance scores below the production threshold. These findings shaped the final design and are documented in Chapter 4.

1.4 Research Objectives and Scope

The project has the following objectives.

1. To build a curated, auditable knowledge base from official CCOE web pages, programme project reports, academic regulations and contact directories.
2. To implement a hybrid retrieval pipeline that combines dense semantic search with BM25 keyword search and reciprocal rank fusion.
3. To generate answers that are grounded in retrieved chunks, with Gemini as the primary model, Groq as failover and a deterministic local extractor as the final fallback.
4. To support multi-turn conversation, programme clarification carry-forward, contact routing and honest refusal behaviour.
5. To deliver a complete web application with authentication, guest access, streaming responses, history, feedback, suggestions and an administration console.

6. To evaluate the system on a question benchmark and document accuracy, latency, robustness and limitations with measured evidence.

The scope of the report covers the four online programmes offered by CCOE, their admissions, fees, eligibility, syllabus, examinations, policies, placements and support contacts. The assistant does not perform admission registration, fee payment, document verification or result publication. It does not answer questions outside the university domain, and it does not replace official authorities. The knowledge base is the boundary of what the system can answer. This boundary is intentional: it is the main mechanism that prevents hallucination.

Table 1.1: Online degree programmes covered by the assistant

Programme	Duration	Mode	Main focus
Online BBA	3 years (6 semesters)	Fully online	Management, accounting and business administration
Online BCA	3 years (6 semesters)	Fully online	Software development, databases and applications
Online MBA	2 years (4 semesters)	Fully online	Finance, marketing, human resources and strategy
Online MCA	2 years (4 semesters)	Fully online	Advanced computing, algorithms and cloud systems

Note: the official documents occasionally report different credit totals or evaluation splits for the same programme. The assistant does not hide these differences. When a conflict is detected, it reports both values together with their sources, as described in Section 4.3.

CHAPTER 2: LITERATURE REVIEW

2.1 Background Study / Related Work

Retrieval-augmented generation was introduced by Lewis et al. as a way to combine a parametric language model with a non-parametric memory that can be inspected and updated [1]. Instead of asking a model to remember every fact, the system retrieves relevant passages at query time and conditions the generation on those passages. This design is particularly suitable for institutional question answering, where facts change and where an incorrect answer has real consequences.

The retrieval component of modern RAG systems usually relies on dense embeddings. Karpukhin et al. showed that a dual-encoder retriever can outperform sparse term matching on open-domain question answering [2]. Dense retrieval maps a question and a passage into the same vector space and ranks passages by cosine similarity. The BGE family of embedding models, including BAAI/bge-small-en-v1.5, provides compact 384-dimensional vectors that run efficiently on CPUs through the ONNX runtime [6], [15]. This property was essential for the project because the free hosting tier does not provide enough memory for a PyTorch-based sentence transformer.

Sparse retrieval remains valuable. BM25 is a probabilistic ranking function that scores a document by term frequency, inverse document frequency and document length normalisation [3]. It is particularly good at exact tokens such as programme codes, course codes and fee headings. Dense retrieval captures meaning and paraphrases but can miss these exact tokens. The two methods are therefore complementary. Reciprocal rank fusion (RRF) combines ranked lists from multiple retrievers by summing a reciprocal function of each rank [4]. It is simple, unsupervised and robust, which makes it a practical choice for a student project with limited labelled data.

Vector databases provide persistent storage and approximate nearest-neighbour search for embeddings. ChromaDB is a lightweight, embedded vector store that can run inside the backend process and persist to a local SQLite file [11]. For a corpus of a few hundred chunks, it removes the operational cost of a separate search service. The broader literature on billion-scale similarity search, such as FAISS, describes the indexing and product-quantisation techniques used by large-scale systems [5]; those techniques become relevant only if the corpus grows by several orders of magnitude.

For answer generation, the project uses hosted large language models. Google Gemini

provides strong instruction following and a generous free tier, while Groq provides very low-latency inference for an open-weight model [7], [8]. A multi-tier design is necessary because free API quotas can be exhausted during demonstrations. The literature on RAG often assumes a reliable model; in practice, rate limits, timeouts and cold starts are part of the engineering problem. The project therefore treats failover and deterministic fallback as first-class requirements rather than afterthoughts.

On the application side, FastAPI provides asynchronous REST endpoints and server-sent events, React provides a component-based user interface, and MongoDB provides a document store for conversations and feedback [9], [10], [12]. These technologies are widely used in production and are well suited to a small team. The project follows the common three-tier pattern: a browser client, an application programming interface and persistent storage, with the retrieval and generation pipeline enclosed inside the backend.

2.2 Summary of Existing Techniques

University chatbots are commonly implemented in one of four ways. The simplest is a rule-based FAQ system with a fixed list of questions and hand-written answers. The second is a fine-tuned language model trained on institutional text. The third is a conventional RAG system that uses only dense vector retrieval. The fourth, adopted in this project, is a hybrid grounded RAG system that combines dense and sparse retrieval with explicit grounding and safety checks. Table 2.1 summarises the trade-offs.

Table 2.1: Comparison of chatbot design approaches

Approach	Retrieval	Grounding	Multi-turn support	Main limitation
Rule-based FAQ	Keyword or menu matching	Hand-written answers only	Limited	Cannot handle paraphrases or new questions
Fine-tuned language model	None (knowledge stored in weights)	Weak	Yes	May hallucinate and becomes stale after policy changes
Dense RAG only	Vector similarity	Document-grounded	Basic	Misses exact codes, numbers and rare terms
Hybrid grounded RAG (proposed)	Dense + BM25 + RRF	Document-grounded with numeric and conflict checks	Context-aware with clarification	Depends on coverage and quality of the document collection

The rule-based approach is cheap and predictable, but it fails as soon as a user phrases a question differently. Fine-tuning can produce natural answers, but it bakes facts into model weights; a fee revision or a new eligibility rule requires retraining, and the model may still invent details. Dense RAG is a strong baseline, but a question such as 'which company recruits' or 'BCA evaluation split' contains exact tokens that are better handled by keyword

search. The hybrid approach addresses this weakness while keeping the transparency of document-grounded answers.

The project also borrows ideas from conversational search. A follow-up such as 'and for MCA?' is ambiguous without the previous turn. Query rewriting, context tracking and clarification questions are therefore part of the retrieval problem, not just the user interface. The implementation keeps a lightweight conversation state and rewrites follow-up questions into standalone queries before retrieval, following the general approach described in recent conversational RAG systems [1], [2].

2.3 Research Gap Identification

The review of existing techniques and the practical audit of the project revealed several gaps that a straightforward RAG implementation does not close.

- Keyword gap: dense embeddings often fail to retrieve exact programme codes, course codes and fee headings, which are common in academic queries.
- Grounding gap: a language model can produce a fluent answer even when the retrieved context does not contain the requested number; a post-generation numeric check is needed.
- Conversation gap: a bare programme name after a fees question is not a new question, but many chatbots treat it as one and lose the original intent.
- Conflict gap: official documents sometimes disagree. A trustworthy assistant should report both values rather than choose one silently.
- Refusal gap: generic fallbacks such as 'I don't know' are not useful; the assistant should explain what is missing and provide the correct official contact.
- Evaluation gap: many student projects report only a demonstration. This project measures accuracy, latency, retrieval recall, concurrency and regression tests on a fixed benchmark.
- Deployment gap: a model that works on a laptop may not fit a free cloud tier. Memory, cold starts and API quotas must be treated as design constraints.

These gaps define the contribution of the project: a hybrid, grounded and auditable assistant with explicit handling of conversation, conflict, refusal and deployment constraints, evaluated with measured results rather than qualitative impressions.

2.4 Problem Formulation

Let C be the curated document collection and let each document be split into a set of chunks D = {d1, d2, ..., dn}. For a user question q with conversation history H, the retrieval stage computes a relevance score s(d, q, H) for every candidate chunk. The score combines the cosine similarity of dense embeddings with the normalised BM25 score, fused by reciprocal rank fusion. The system keeps the top k chunks whose true cosine relevance is at least a threshold tau, with k = 6 and tau = 0.52 in the production configuration.

Formally, the retrieved context is R = {d in top-k(C, q, H) : cos(d, q) >= tau}. If R is empty, the system returns an honest refusal with official contact information. Otherwise, the generator G produces an answer a = G(q, H, R) under a prompt that forbids unsupported facts. A verification function V(a, R) checks that every numeric token in the answer appears in R or in a permitted arithmetic combination of numbers in R. If V fails, the answer is regenerated or replaced by a safe refusal. The objective is to maximise the number of correct and grounded answers while minimising wrong answers, false refusals and response latency.

For clarification, let p be the set of programmes mentioned in q and H. If the question requires a programme but p is empty, the system returns a clarification question with four programme options and stores the original question q0, the topic and the missing slot. When the user replies with a programme or a set of programmes, the system answers q0 for that programme set instead of treating the reply as a new question. This behaviour is formalised as a small slot-filling step that runs before retrieval.

CHAPTER 3: METHODOLOGY

3.1 Overview of the Proposed Work

The proposed system is a three-tier web application with an offline indexing pipeline and an online answering pipeline. The offline pipeline converts official web pages and PDF documents into clean, overlapping text chunks, generates embeddings and stores them in a persistent vector database. The online pipeline accepts a user question, understands the conversation context, retrieves relevant chunks, generates a grounded answer and streams it back to the browser. MongoDB stores user accounts, conversations, feedback and administrative audit events.

The design follows four principles. First, the document collection is the single source of truth. Second, retrieval combines semantic and lexical signals so that both paraphrases and exact codes are handled. Third, generation is constrained by a strict prompt and post-generation checks. Fourth, every uncertain case fails safely with an honest refusal and a useful official contact. These principles are visible in the architecture and in the behaviour of the deployed assistant.

The implementation is modular. Each stage of the pipeline is a separate Python module with a narrow responsibility: scraping, PDF parsing, chunking, embedding, retrieval, conversation analysis, prompt construction, model invocation, conflict detection and text cleanup. This separation made it possible to test individual stages offline, to replace the embedding model or the language model without rewriting the pipeline, and to measure the effect of each change on the benchmark.

3.2 System / Framework Design

Figure 3.1 shows the end-to-end architecture. The browser client communicates with the FastAPI backend over HTTPS. Authenticated requests carry a JSON Web Token, while guest requests use a short-lived guest session with a four-message limit. The backend runs the query-understanding stage, the hybrid retriever, the grounded generation tier and the safety checks. MongoDB Atlas stores users, conversations and feedback. The offline indexing path is shown at the bottom: official web pages and PDF files are scraped or parsed, cleaned, chunked, embedded and written to ChromaDB.

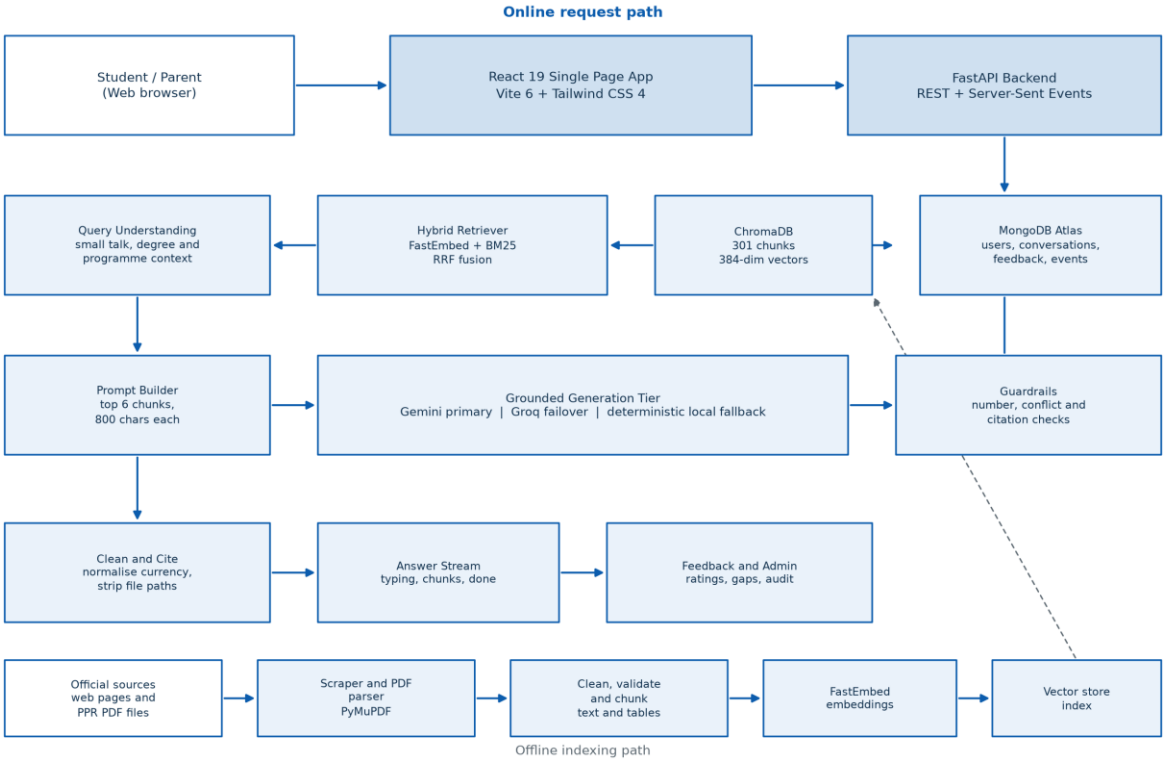


Figure 3.1: End-to-end system architecture of the CHARUSAT Online Course Assistant

The frontend is a React 19 single-page application built with Vite 6 and styled with Tailwind CSS 4. It provides a chat-first interface with light and dark themes, a responsive sidebar for conversation history, a composer with as-you-type suggestions, streamed assistant messages, programme buttons for clarification, feedback controls and a settings page. The administration console is a separate protected area with dashboards for usage, knowledge gaps, conversations, feedback and users.

The backend is a FastAPI application. The chat routes provide both a normal JSON endpoint and a server-sent events endpoint that emits typing, chunk, done and error events. Authentication routes handle email and password signup, login, Google identity login and profile updates. Conversation routes provide create, list, rename and delete operations. Admin routes provide statistics, gap review, user management, feedback review and CSV export. A separate suggestion route powers the as-you-type question suggestions, and a health endpoint reports the status of the vector store, the database and the language-model clients.

The retrieval layer uses FastEmbed with the BAAI/bge-small-en-v1.5 model. The model produces 384-dimensional vectors and runs on the ONNX runtime, which keeps memory usage close to 100 MB. ChromaDB persists the vectors in backend/vector_db. BM25 is implemented with rank-bm25 and an in-memory index built from the same chunks. Reciprocal rank fusion merges the two ranked lists, while the original cosine similarity is preserved for the final relevance threshold. Keeping the true cosine score separate from the

fused rank was an important fix: early versions applied the threshold to the fused score, which compressed all scores to a small range and caused valid questions to return no chunks.

The generation layer uses a priority list of Gemini models, with Groq's openai/gpt-oss-120b as the failover. If both providers are unavailable, a deterministic local extractor selects grounded sentences from the retrieved chunks. The answer is then cleaned and checked. Currency notation is normalised, file paths and internal references are removed, conflicting values are reported with their sources, and numeric tokens that do not appear in the context are flagged. The final answer is streamed to the client and stored in the conversation.

Table 3.1: Technology stack

Layer	Technologies	Role
Client	React 19, Vite 6, Tailwind CSS 4, react-router-dom 7, react-markdown, remark-gfm	Responsive chat interface, themes, streamed messages and Markdown tables
API	FastAPI, Uvicorn, Pydantic, python-dotenv	REST endpoints, server-sent events, validation and configuration
Retrieval	FastEmbed, BAAI/bge-small-en-v1.5, ChromaDB, rank-bm25, reciprocal rank fusion	Dense and sparse search with fused ranking
Generation	Google Gemini, Groq openai/gpt-oss-120b, deterministic local extractor	Grounded answer generation and multi-tier failover
Data	MongoDB Atlas, PyJWT, bcrypt	Users, conversations, feedback, audit events and security
Ingestion	requests, BeautifulSoup, crawl4ai, PyMuPDF, LangChain text splitters	Web scraping, PDF parsing, cleaning and chunking
Deployment	Render, Vercel, GitHub	Backend hosting, frontend hosting and source control
Testing	pytest, Node test runner, Playwright, custom rule scanner	Regression, browser and compliance checks

3.3 Methodology / Workflow

Figure 3.2 shows the online query workflow. The sequence is deliberately linear, with two early exits: a polite reply for greetings or out-of-scope questions, and an honest refusal when retrieval finds no chunk above the relevance threshold. The clarification branch is entered only when a programme is required but missing. This keeps the common path fast and makes the exceptional paths easy to test.

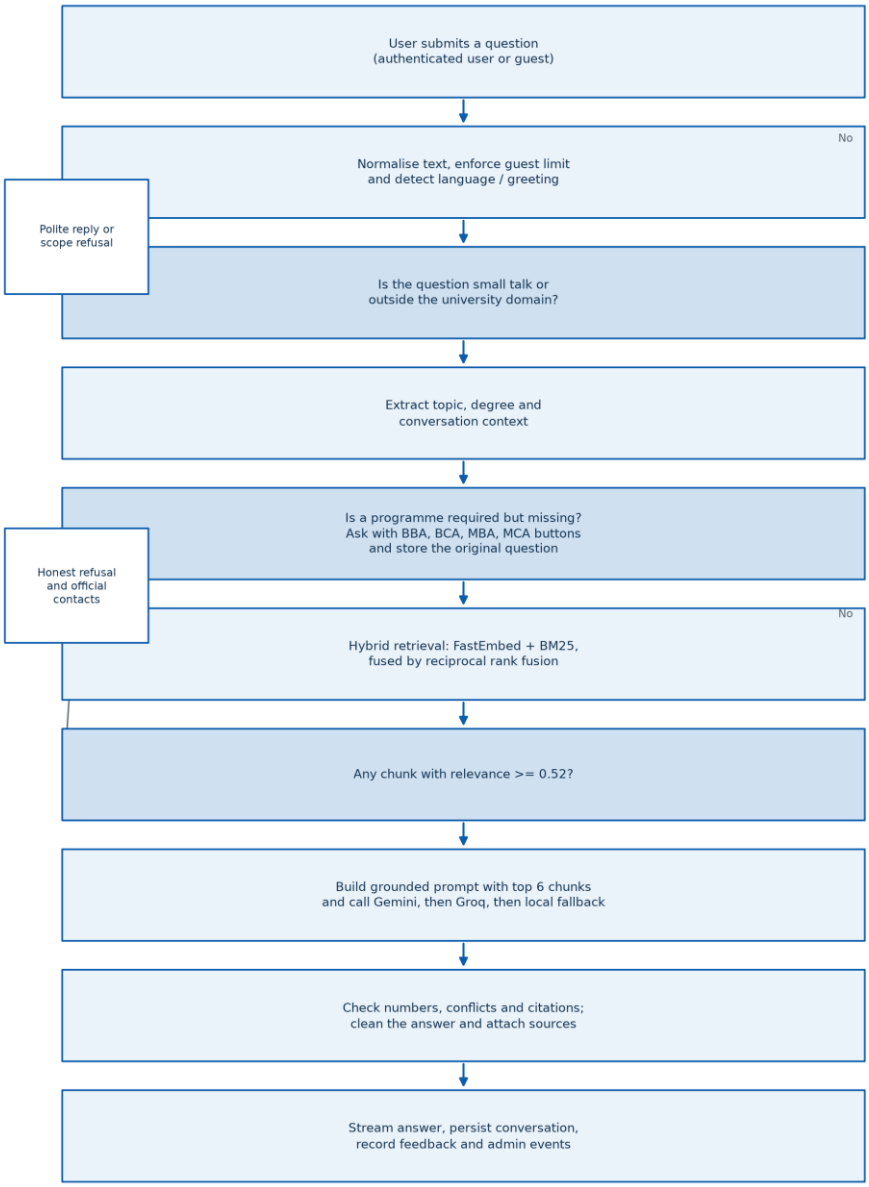


Figure 3.2: End-to-end query workflow with clarification, retrieval and refusal branches

3.3.1 Offline indexing pipeline

The indexing pipeline begins with the official CCOE website and the programme project reports. The scraper starts at the home page, follows internal links, removes navigation and banners, and saves clean Markdown with source URL, title and timestamp metadata. The PDF ingestion module uses PyMuPDF to extract text and tables from the four programme project reports and the refund policy. Tables are converted to Markdown, empty columns and rows are removed, and repeated headers are merged so that fee rows remain readable.

The cleaner removes zero-width characters, non-breaking spaces, duplicated prefixes and empty tables. Chunks shorter than 80 characters or with fewer than 40 alphanumeric

characters are discarded. Exact duplicates are removed, while cross-programme repeated requirements are preserved because they carry different programme metadata. Each valid chunk stores the source file, page title, programme name, document type, source URL and last-scraped timestamp. The embeddings are computed with FastEmbed and written to ChromaDB together with this metadata.

Table 3.2: Chunking configuration

Document type	Splitter	Chunk size	Overlap	Notes
Web Markdown	Markdown and recursive character splitter	1000	200	Headings and paragraphs preserved
PDF text	Recursive character splitter	1200	200	Separated from table segments
PDF tables	Table-aware splitter	4000	100	Pipe separator keeps fee rows together
Validation	Custom filters	Minimum 80 characters	Not applicable	Empty tables, short chunks and duplicates removed

3.3.2 Online query pipeline

When a question arrives, the backend first normalises the text and checks the authentication state. Guests may send four messages before the interface invites them to sign in. The small-talk module detects greetings, thanks and out-of-scope requests, but it is guarded by a course-keyword check so that a greeting followed by a real question is not misclassified. The conversation module extracts the topic, programme, year and intent, and rewrites a follow-up into a standalone query when the previous turn supplies the missing context.

The query is then expanded with topic synonyms from the lexicon. For example, placement queries are expanded with terms such as recruiter, company and recruitment; examination queries are expanded with transcript, marksheet and result. The hybrid retriever runs dense search and BM25 search in parallel, fuses the ranked lists with reciprocal rank fusion, and keeps the best six chunks above the 0.52 cosine threshold. If a programme-independent topic is detected, such as refunds or examinations, the search is broadened across programmes rather than being restricted to the programme of the previous turn.

The prompt builder wraps the chunks in a strict instruction template. The template tells the model to answer only from the provided context, to preserve exact numbers and programme names, to use Markdown tables for comparisons, to state both values when documents conflict, and to say that a detail is not in the documents instead of guessing. The model is invoked through the Gemini client first. If Gemini returns a rate-limit error or times out, the Groq client is used. If both fail, the local extractor returns the most relevant grounded

sentences.

The final stage performs post-generation checks. The conflict detector compares numeric claims across sources. The numeric verifier checks that every number in the answer appears in the retrieved context or is a permitted arithmetic total. The cleanup module normalises currency, removes internal file names and fixes broken spacing. The answer is then streamed to the client as a sequence of events and stored in MongoDB. Feedback, latency, retrieval metadata and gap status are recorded for the administration console.

3.3.3 Programme clarification carry-forward

A common failure in conversational assistants is to ask 'Which programme?' and then forget the original question. The project treats clarification as a small slot-filling problem. When the user asks a question such as 'What is the fee structure?' without a programme, the assistant replies with four rectangular buttons for BBA, BCA, MBA and MCA. The assistant message carries metadata that contains the original question, the topic and the missing slot. The client sends this metadata back with the next request. If the client is a guest and has no server state, the backend reconstructs the clarification from the conversation history.

A short reply such as 'bca', 'BCA please', 'for mca', 'online bca' or 'bca and mca' resolves the clarification. The assistant then answers the original question for the selected programme or programmes. A reply of 'all' or 'any' resolves to all four programmes. A new complete question is treated as a new turn and does not inherit the clarification. A reply that is neither a programme nor a question triggers the clarification once more with the buttons. This behaviour is covered by offline regression tests.

3.3.4 Suggestions, streaming and safety

The as-you-type suggestion feature uses a verified bank of 117 questions. The client sends the current prefix to the suggestion endpoint after a short debounce. The backend ranks candidates from the question bank and, when available, asks a small language model to paraphrase them. Results are filtered for length, duplication, URL content and knowledge gaps. If the models are unavailable, the endpoint returns the nearest bank questions, so the feature degrades gracefully instead of failing.

The chat endpoint streams the answer over server-sent events. The client shows a typing indicator, renders chunks as they arrive, supports a stop control with AbortController, and displays a calm notice if the free backend is waking from a cold start. Safety rules are applied on both sides: the backend never returns raw file paths or student records, and the frontend

never displays unverified claims as official facts. The design system also enforces a consistent blue palette, rectangular controls, light and dark themes and a 360-pixel mobile layout.

3.4 Tools, Technologies, or Environment Used

The project was developed on Windows with Python and Node.js. The backend code targets Python 3.11 for compatibility with the Render runtime, while local development and testing were performed with Python 3.14.6. The frontend uses Node.js with Vite. MongoDB Atlas provides the production database, and ChromaDB is embedded in the backend process. Table 3.3 summarises the environment.

Table 3.3: Development, testing and deployment environment

Item	Value
Development operating system	Windows 11
Backend runtime	Python 3.11 compatible code; local tests on Python 3.14.6
Frontend runtime	Node.js with Vite 6
Vector store	ChromaDB persisted in backend/vector_db
Database	MongoDB Atlas
Language models	Gemini 3.6, 3.5 and 2.5 Flash, Gemini 2.5 Pro, Groq gpt-oss-120b
Embedding model	BAAI/bge-small-en-v1.5 through FastEmbed (ONNX, 384 dimensions)
Backend hosting	Render free web service running Python 3.11.9
Frontend hosting	Vercel static deployment with SPA rewrites
Testing	pytest, Node test runner, Playwright and a custom rule scanner

The deployment uses two services. Render runs the FastAPI application from the deploy branch with the command uvicorn api:app --host 0.0.0.0 --port $PORT. Vercel builds the frontend with npm run build and serves the dist directory with a rewrite to index.html for client-side routing. Environment variables hold the model keys, the MongoDB URI, the JWT secret and the frontend origin. No key is stored in the repository or in the frontend bundle.

CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1 Implementation Details

The implementation is organised around the knowledge base, the backend modules, the frontend modules and the public API. The knowledge base currently contains 24 curated Markdown documents. Twenty-three of these are indexed into ChromaDB, producing 301 chunks. The placement recruiter addendum is kept as an unmanaged document so that it can be updated without rebuilding the main index. Table 4.1 summarises the composition and the number of indexed chunks per category.

Table 4.1: Knowledge base composition and indexed chunks

Category	Documents	Indexed chunks
Programme overview pages	online_bba_comprehensive.md, online_bca_comprehensive.md, online_mba_comprehensive.md, online_mca_comprehensive.md	69
Programme project reports	ppr_online_bba.md, ppr_online_bca.md, ppr_online_mba.md, ppr_online_mca.md	94
Academic and examination rules	terms-conditions.md, refund_policy.md, privacy-policy.md, admission_cancellation.md, online_examination_comprehensive.md, aicte_eoa_comprehensive.md, ugc_public_notice.md, ciqa.md	81
Syllabi	bca_syllabus_2025_26.md, mca_syllabus_2025_26.md	15
Admissions	admissions_guide.md	9
Contacts and faculty	contact_directory.md, faculty.md	14
Home page	home.md	17
Feedback	feedback.md	2
Placement addendum	placement_recruiters.md	Not indexed (unmanaged addendum)
Total	24 documents (23 indexed)	301

The backend modules follow the pipeline stages. Table 4.2 lists the principal modules and their responsibilities. The separation is important for testing: retrieval, conversation analysis, prompt construction, model invocation and post-processing can each be exercised without a live API call.

Table 4.2: Principal backend modules

Module	Responsibility
api.py	FastAPI application, health checks and public endpoints
routes/chat_routes.py	Chat, streaming, feedback and guest migration
routes/auth_routes.py	Signup, login, Google login and profile updates
routes/conversation_routes.py	Conversation create, list, rename and delete
routes/admin_routes.py	Statistics, knowledge gaps, users, feedback and exports
routes/suggest_routes.py	As-you-type question suggestions

Module	Responsibility
config.py	Thresholds, model names, paths and runtime settings
scraper.py	Crawl the official website into clean Markdown
ingestion.py	Clean, chunk and index web text
pdf_ingestion.py	Extract text and Markdown tables from PDF files
embeddings.py	FastEmbed encoder and ChromaDB access
retriever.py	Routing, candidate search and relevance filtering
hybrid_retrieval.py	BM25 index, reciprocal rank fusion and score preservation
conversation.py	Turn analysis, context tracking and follow-up rewriting
lexicon.py and topic_lexicon.json	Topic vocabulary, synonyms and programme mapping
smalltalk.py	Greetings, off-topic detection and scope handling
contacts.py	Official department contact routing
prompt.py	Grounded system prompt and answer rules
llm.py	Gemini, Groq and deterministic local fallback
rag_chain.py	Pipeline orchestration, numeric checks and conflict handling
auth.py, database.py, models.py	Security, persistence and data models
suggest_service.py, cache.py, text_cleanup.py	Suggestions, caching and text normalisation

The frontend implements the user-facing side of the same design. Table 4.3 lists the principal pages and components. The chat page is the default route. The settings page exposes profile customisation, theme selection, data export, history clearing and a tips toggle. The admin console is protected by an administrator role and provides five tabs: overview, knowledge gaps, conversations, feedback and users.

Table 4.3: Principal frontend modules

Module	Responsibility
App.jsx	Routing, theme provider and error boundary
pages/Chat.jsx	Main chat experience and conversation view
pages/Login.jsx and Signup.jsx	Email and Google authentication
pages/Settings.jsx	Profile, theme, export and chat preferences
pages/Admin.jsx	Five-tab administration console
pages/About.jsx	Factual description of the assistant and official contacts
components/chat/*	Composer, message list, sidebar, welcome screen and suggestions
components/ui/*	Dialog, toast, theme toggle and branded logos
hooks/useChat.js	Streaming state, abort control and error recovery
hooks/useSidebar.js	Conversation navigation and management
contexts/AuthContext.jsx and ThemeContext.jsx	Session and theme state
services/api.js, chat.js and auth.js	Backend communication

Module	Responsibility
lib/questionBank.js, welcomeHelper.js and tipSelector.js	Suggestions, greetings and tips

The API surface is intentionally small. Table 4.4 lists the endpoints used by the client and the administration console. All protected routes require a valid JWT. Guest chat requests are accepted with a guest identifier and a message counter. Admin routes additionally check the administrator role and record sensitive actions in an audit log.

Table 4.4: Public and protected API endpoints

Endpoint	Purpose
POST /api/chat	Answer a question in a single JSON response
POST /api/chat/stream	Stream typing, chunk, done and error events
GET, POST, PATCH, DELETE /api/conversations	List, create, rename and delete conversations
POST /api/chat/feedback	Record a positive or negative rating and an optional correction
POST /api/chat/migrate	Move a guest conversation to an authenticated account
POST /api/suggest	Return as-you-type question suggestions
GET /api/question-bank	Return verified starter questions
GET /api/contacts	Return official department contacts
GET and PATCH /api/me	Read and update the user profile
POST /api/auth/*	Signup, login, Google login and session handling
GET /api/admin/*	Dashboard, gaps, users, feedback and CSV exports
GET /health	Report the status of the API and its dependencies

Authentication uses bcrypt password hashing and JSON Web Tokens. The guest mode allows four messages before the interface invites the visitor to sign in. When a guest signs in, the migration endpoint moves the guest conversation to the new account. Conversation titles are generated from the first question and can be renamed. The feedback endpoint stores a rating and an optional correction against the assistant message. The admin console aggregates these events into usage charts and knowledge-gap tables without exposing raw personal data by default.

The suggestion service is a separate implementation from answer generation. It uses the verified question bank as a fallback and a short language-model call for paraphrasing. The service enforces a strict length limit, removes URLs and long digit sequences, requires a question mark, removes duplicates and filters questions that are known to be knowledge gaps. A retriever gate checks that the suggested question is answerable within a small time budget. Rate limits are applied separately for authenticated users and guests.

Text cleanup is applied to every answer. The cleaner removes zero-width characters, normalises non-breaking and narrow spaces, converts lone em dashes in tables to 'N/A', converts prose em dashes to commas, and removes internal file references. These rules were added after users reported words such as 'CHARUSATOnline' and 'the homepage' being glued together by zero-width characters copied from the source pages. The cleanup is applied on the fly during reads, so the stored conversation is not rewritten.

4.2 Performance Evaluation Metrics

The system was evaluated with a 77-question benchmark that covers all four programmes and the main policy areas. Each answer was classified as correct, correct refusal, wrong answer or execution error. A correct refusal is counted as accurate when the requested fact is genuinely absent from the knowledge base. A wrong answer includes a false refusal when the fact is present, a wrong number, an unsupported claim or a failure to answer the question. Latency was measured from the HTTP request to the complete response. Component tests measured retrieval recall, contact routing, concurrency, cache separation and numeric conflict detection.

Table 4.5 compares the Round 5B baseline with the final Round 5C measurements. The final run was performed in three modes: a single conversation with cache enabled, a single conversation with cache disabled, and a fresh conversation for each question. The three modes produced consistent results, which rules out cache or conversation state as the source of the improvement.

Table 4.5: Benchmark answer quality on 77 questions

Metric	Round 5B baseline	Round 5C, cache on	Round 5C, cache off
Questions evaluated	77	77	77
Correct answers	35	39	38
Correct refusals	15	19	19
Total accurate answers	50 (64.9%)	58 (75.3%)	57 (74.0%)
Wrong or false refusal	27	19	20
Execution errors	0	0	0

Table 4.6 reports the latency distribution. The baseline was affected by sequential failover across exhausted Groq models, which caused timeouts of up to 46 seconds. The final version detects organisation-level token-limit errors immediately, places the provider on a short cooldown and moves to the next option without waiting for a full timeout. Removing an Arabic model that was slow on long English prompts also reduced the worst case.

Table 4.6: End-to-end latency

Metric	Round 5B baseline	Round 5C, cache on	Round 5C, cache off
50th percentile	18.5 s	0.714 s	0.684 s
90th percentile	24.6 s	0.836 s	0.772 s
Maximum	46.4 s	2.549 s	0.938 s

Component tests provide a second view of reliability. Table 4.7 lists the measured results. The concurrency tests are important because the retriever and the pipeline were originally written with shared mutable state. The final implementation uses local variables and a structured result object, so simultaneous requests do not mix programmes or topics.

Table 4.7: Component and stress-test results

Test	Scope	Result
Contact routing	30 paraphrases across six services	30/30 passed
Real-path concurrency	20 parallel chat requests	20/20 passed, 0.52 to 4.31 s
Cache separation	11 sequential fee and admission questions	11/11 passed
Conflict and number checks	6 unit checks	6/6 passed
Small-talk edge cases	8 greeting and typo cases	8/8 passed
Typo paraphrase set	80 paraphrases across four programmes	80/80 passed
New paraphrase set	20 unseen paraphrases	19/20 passed
Parallel stress	10 parallel requests	9/10 passed
R1 compliance scan	Backend and frontend source	0 violations

The automated suite was executed on the final branch. The backend offline suite passed 56 tests, the frontend unit suite passed 12 tests, and the Playwright browser suite passed 11 tests. The custom rule scanner reported no violations, and the production build completed successfully. These results are summarised in Table 4.8.

Table 4.8: Automated test and build results

Suite	Command	Result
Rule scanner	npm run check:rules	All clean, 0 errors
Production build	npm run build	339 modules transformed, success
Frontend unit tests	node --test tests/unit/*.test.js	12 passed, 0 failed
Browser tests	npm run test:e2e	11 passed, 0 failed
Backend offline tests	pytest -q	56 passed in 35.15 s

The benchmark and component results show that the system is reliable for the tested question set, but they do not prove that it will answer every possible question. The knowledge base remains the limiting factor. Questions about future admission deadlines, faculty profiles, hostel facilities or recruitment schedules are correctly refused because those facts are not in the documents. This is the intended behaviour, not a defect.

4.3 Result Analysis and Discussion

Figure 4.1 visualises the two most important results. The left panel shows that the number of correct answers increased from 35 to 39 and the number of wrong or false-refusal answers decreased from 27 to 19. The right panel shows the latency improvement on a logarithmic scale. The 90th percentile latency dropped from 24.6 seconds to 0.836 seconds, which is well below the 15-second target that was set at the beginning of the optimisation round.

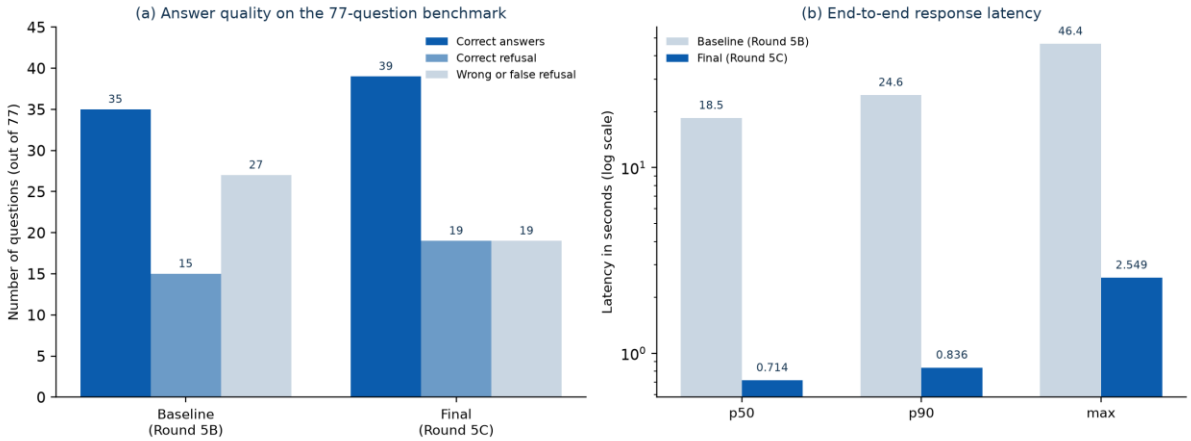


Figure 4.1: Answer quality and latency before and after the final optimisation round

The accuracy improvement came from several small fixes rather than one large change. First, the retrieval pipeline stopped applying the relevance threshold to the fused RRF score and started applying it to the true cosine similarity. Second, the topic lexicon gained synonyms for refunds, placements, transcripts, attendance and technical support. Third, the refusal formatter changed from prepending a negative sentence to adding a secondary note when most of the answer was present. Fourth, the numeric checker learned Indian number formats such as lakh and crore and accepted arithmetic totals. Fifth, the prompt began to report conflicting values from different official documents instead of choosing one silently.

The retrieval recall result is particularly important. In an earlier audit, 19 of 40 benchmark questions returned no chunks because the RRF score range was compressed below the threshold. After the score separation fix, all 40 questions returned chunks. This shows that a single design mistake in the scoring layer can make a correct document collection look useless. The lesson is that retrieval quality must be measured at the chunk level, not inferred from the final answer alone.

Table 4.9: Before and after improvements

Area	Before	After	Cause of change
Retrieval recall	19 of 40 queries returned no chunks	40 of 40 returned chunks	Separated fused rank from true cosine score

Area	Before	After	Cause of change
Answer accuracy	50 of 77 accurate (64.9%)	58 of 77 accurate (75.3%)	Better grounding, lexicon and refusal formatting
Wrong or false refusal	27 questions	19 questions	Cleaner retrieval and refusal logic
90th percentile latency	24.6 s	0.836 s	Fast break on provider quota errors
Backend memory	About 520 MB, out of memory on Render	About 110 MB	FastEmbed ONNX replaced PyTorch
Cross-degree leakage	BCA queries returned MCA chunks	0% measured leakage	Metadata filtering in BM25
Question bank	Unverified questions	117 validated questions	Offline validation against documents

The question bank provides a complementary measure of coverage. Of the 117 validated questions, 92 (78.6 percent) have a full answer in the documents, 21 (17.9 percent) have a partial answer, and 4 (3.4 percent) are known to be absent and trigger a verified fallback. Table 4.10 summarises the distribution. The bank is used for starter questions and as-you-type suggestions, so it also communicates the scope of the assistant to users.

Table 4.10: Question bank coverage

Category	Count	Share	Meaning
FULL	92	78.6%	The answer is present in official documents
PARTIAL	21	17.9%	Part of the answer is available
NOT_IN_DOCS	4	3.4%	A verified fallback is required
Total	117	100%	Validated starter bank

Residual weaknesses remain. The first is multi-programme synthesis. When a user asks for a comparison across all four programmes, the top six retrieved chunks can all come from one programme, so the answer may cover only that programme. The second is multilingual retrieval. The embedding model is English-only, so a fully Gujarati or Hindi query may retrieve a general programme page instead of the exact fee table, even though the answer can be written in the same language. The third is out-of-scope token overhead: a question about foreign exchange or internships can retrieve general promotional text because it shares words such as CHARUSAT, degree and online. The fourth is chunk boundary splitting: fixed-size chunking can cut a sentence in the middle. The fifth is the free hosting tier, where a cold start can add 20 to 40 seconds to the first request. These limitations are documented so that future work can address them directly.

The project also exposed several knowledge base contradictions. For example, some documents report different examination splits or credit totals for the same programme, and one marketing page contains an eligibility line copied from another programme. The assistant

does not hide these conflicts. The conflict detector identifies the disagreement and the prompt instructs the model to report both values with their sources. This is a more honest behaviour than selecting one value and presenting it as certain.

4.4 Comparison with Existing Methods

Table 4.11 compares the proposed system with the common alternatives discussed in Chapter 2. The comparison uses criteria that matter to an academic institution: factual grounding, handling of paraphrases and typos, exact codes and numbers, source transparency, conversational context, hallucination control, latency and maintenance effort. The values for the proposed system are based on the measurements reported in this chapter; the values for the alternatives are qualitative assessments based on their design.

Table 4.11: Comparison with existing approaches

Criterion	Rule-based FAQ	Fine-tuned model	Dense RAG	Proposed hybrid RAG
Factual grounding	High for written answers	Low	High	High
Paraphrases and typos	Low	High	High	High
Exact codes and numbers	Medium	Low	Medium	High
Source transparency	None	None	Partial	Full internal citations
Multi-turn context	Limited	Yes	Basic	Yes, with clarification
Hallucination control	By design only	Weak	Medium	Numeric and conflict checks
Measured latency	Very low	Low	Moderate	0.836 s at p90
Maintenance	High manual effort	Retraining required	Re-index documents	Re-index documents

The comparison shows that the proposed system is not the cheapest option in terms of engineering effort. It requires a document pipeline, a retrieval layer, a generation tier, safety checks and a web application. However, it provides the combination that the institution needs: answers that are grounded in official documents, tolerant of real student language, transparent enough to audit and safe enough to deploy. A rule-based FAQ cannot handle paraphrases, and a fine-tuned model cannot guarantee that a fee figure is current. The hybrid RAG design is the best fit for a document-bound academic assistant.

CHAPTER 5: CONCLUSION AND FUTURE EXTENSIONS

5.1 Summary of Contributions

This project designed, implemented and evaluated a complete retrieval-augmented generation assistant for the online degree programmes of CHARUSAT CCOE. The system covers the four programmes, uses a curated knowledge base of official documents, combines dense and sparse retrieval, generates grounded answers with a multi-tier model fallback and exposes the result through a production-style web application. The work makes the following contributions.

- A reproducible document pipeline that converts official web pages and PDF files into a clean, metadata-rich vector index of 301 chunks.
- A hybrid retrieval design that combines FastEmbed semantic search and BM25 keyword search with reciprocal rank fusion and preserves the true cosine score for thresholding.
- A grounded generation tier with Gemini, Groq and deterministic local fallback, plus numeric, conflict and citation checks.
- A conversational layer with topic and programme context, follow-up rewriting, programme clarification carry-forward and official contact routing.
- A complete web application with streaming chat, authentication, guest mode, history, suggestions, feedback, settings and an administration console.
- A measured evaluation on a 77-question benchmark, with accuracy, latency, recall, concurrency and regression results, plus an honest account of the remaining limitations.

The measured results support the design. Accurate answers increased from 64.9 percent to 75.3 percent, wrong or false-refusal answers decreased from 27 to 19, and the 90th percentile latency fell from 24.6 seconds to 0.836 seconds. The final suite passes 56 backend tests, 12 frontend unit tests and 11 browser tests, and the rule scanner reports no violations.

5.2 Limitations

The system has several limitations that should be considered before a wider public launch.

- The knowledge base is the boundary. If a fact is not in the official documents, the assistant will refuse it, even when the fact is true.
- Multi-programme comparison questions can retrieve chunks from only one programme

because the top six slots are limited.

- The English-only embedding model retrieves poorly for fully Gujarati or Hindi queries.
- Official documents sometimes conflict, and the assistant can only report the conflict, not resolve which document is authoritative.
- The free hosting tier introduces cold-start delays and limits sustained traffic.
- The assistant does not process admissions, payments or document verification, and it does not replace official staff.
- The benchmark is large enough for a student project but does not cover every possible phrasing or every programme policy.

These limitations are not hidden in the interface. The About page explains that answers come from official documents and that the assistant can be wrong or incomplete. The refusal messages provide official contact details so that the user can reach a human when the document collection is not enough.

5.3 Future Extensions

Several extensions would improve coverage, accuracy and cost efficiency.

- Merge the completed feature branch into the deployment branch and run the migration and vector-index rebuild in the production environment.
- Schedule a weekly crawl and document validation job so that fee changes, examination notices and admission dates are kept current.
- Build the vector index in the deployment pipeline instead of committing binary index files.
- Replace the English-only embedding model with a multilingual model such as BGE-M3 for Gujarati and Hindi queries.
- Add a programme-aware retrieval strategy that reserves slots for each programme in comparison questions.
- Train or configure a domain-specific reranker for academic policy text, and evaluate it against the current hybrid baseline before enabling it.
- Add a feedback-to-knowledge loop in which repeated knowledge gaps are reviewed by staff and converted into official documents.
- Evaluate local open-weight model hosting to reduce dependence on free API quotas and

to remove external rate limits.

- Add automated accessibility, load and security tests to the continuous integration pipeline.

The most valuable short-term extension is not a new model. It is a reliable document operations process: who updates the knowledge base, how changes are validated, and how the assistant reports gaps back to the institution. A RAG assistant is only as good as the documents behind it, and a maintained document process is more valuable than a larger language model.

5.4 Conclusion

The CHARUSAT Online Course Assistant demonstrates that a small student team can build a trustworthy academic chatbot with open technologies. The project combines document engineering, hybrid retrieval, grounded generation, conversational state and full-stack web development in a single system. It does not pretend to know everything. It answers from official documents, shows its evidence internally, reports conflicts, refuses unsupported questions and directs users to the right official contact. That restraint is the most important design decision in the project.

The measured results confirm that the approach is practical. The system answers the majority of the benchmark accurately, responds in under one second at the 90th percentile after optimisation and passes a broad regression suite. The remaining limitations point to clear future work: multilingual retrieval, multi-programme synthesis, domain reranking and a maintained document pipeline. With those improvements, the assistant can become a dependable first point of contact for prospective and current online learners at CHARUSAT.

BIBLIOGRAPHY

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020, arXiv:2005.11401.

[2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769-6781.

[3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009.

[4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in Proc. International ACM SIGIR Conference, 2009, pp. 758-759.

[5] J. Johnson, M. Douze, and H. Jegou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2021.

[6] Beijing Academy of Artificial Intelligence, "BAAI/bge-small-en-v1.5 Model Card," Hugging Face, 2023. [Online]. Available: https://huggingface.co/BAAI/bge-small-en-v1.5

[7] Google, "Gemini API Documentation," 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs

[8] Groq, "GroqCloud Documentation," 2026. [Online]. Available: https://console.groq.com/docs

[9] S. Ramirez, "FastAPI Documentation," 2026. [Online]. Available: https://fastapi.tiangolo.com

[10] Meta Open Source, "React Documentation," 2026. [Online]. Available: https://react.dev

[11] Chroma, "Chroma Documentation," 2026. [Online]. Available: https://docs.trychroma.com

[12] MongoDB, "MongoDB Atlas Documentation," 2026. [Online]. Available: https://www.mongodb.com/docs/atlas/

[13] Render, "Deploy a FastAPI Application," 2026. [Online]. Available: https://render.com/docs/deploy-fastapi

[14] Vercel, "Deploying Vite Applications," 2026. [Online]. Available: https://vercel.com/docs/frameworks/vite

[15] Qdrant, "FastEmbed Documentation," 2026. [Online]. Available: https://qdrant.github.io/fastembed/

[16] Charotar University of Science and Technology, "Center for Online Education: Programmes," 2026. [Online]. Available: https://charusat.online/